

Capacity, Overfitting and Underfitting

- The ability to perform well on previously unobserved inputs is called **generalization**.
- when training a machine learning model, we have access to a **training set**, we can compute some error measure on the training set called the **training error**, and we **reduce this training error**.
- This is an **optimization problem**.
- What separates machine learning from optimization is that we want the **generalization error, also called the test error, to be low** as well.
- **generalization error** is the expected value of error on a **new input**

Capacity, Overfitting and Underfitting

- we trained the model by minimizing the training error,
- $\frac{1}{m^{(train)}} \| \mathbf{X}^{(train)} \mathbf{w} - \mathbf{y}^{(train)} \|_2^2$

but we actually care about the test error,

- $\frac{1}{m^{(test)}} \| \mathbf{X}^{(test)} \mathbf{w} - \mathbf{y}^{(test)} \|_2^2$

How can we affect performance on the test set when we get to observe only the training set?

Capacity, Overfitting and Underfitting

- The train and test data are generated by a probability distribution over datasets called the data generating process.
- We typically make a set of assumptions known collectively as the **i.i.d. assumptions**.
- These assumptions are that the examples in each dataset are independent from each other,
- and that the **train set and test set** are **identically distributed**, drawn from the same probability distribution as each other.

Capacity, Overfitting and Underfitting

- The factors determining how well a machine learning algorithm will perform are its ability to:
 1. **Make the training error small.**
 2. **Make the gap between training and test error small.**
- These two factors correspond to the two central challenges in machine learning:
- **underfitting and overfitting .**
- **Underfitting occurs** when the model is not able to obtain a sufficiently **low error value on the training set.**
- **Overfitting occurs** when the gap between the **training error and test error** is too large.

Capacity, Overfitting and Underfitting

- A model is said to be a good machine learning to make predictions about future data.
- overfitting and underfitting, which are majorly responsible for the **poor performances**.

underfitting :

- a model is too simple to capture data complexities.
- inability of the model to learn the training data effectively result in poor performance **both on the training and testing data**.
- Models are inaccurate
- size of the training dataset used is not enough.

Overfitting

- model does not make accurate predictions on testing data.
- When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set.

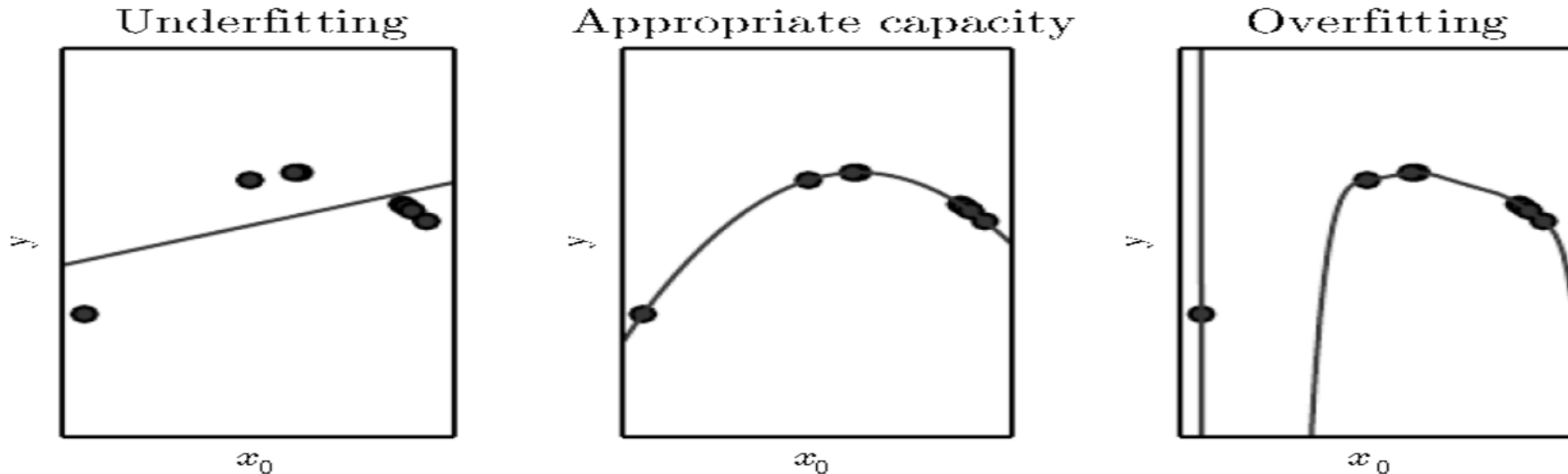
Capacity, Overfitting and Underfitting

- We can control whether a model is more likely to overfit or underfit by **altering its capacity**.
- Models with **low capacity** may struggle to fit the training set.
- Models with **high capacity** can overfit by memorizing properties of the training set that do not serve them well on the test set.
- We can generalize linear regression to include polynomials, rather than just linear functions, in its hypothesis space. Doing so increases the model's capacity.
- $\hat{y} = b + wx$
- $\hat{y} = b + w_1x + w_2x^2$
- obtain a polynomial of degree 9:
- $\hat{y} = b + \sum_{i=1}^9 w_i x^i$

Capacity, Overfitting and Underfitting

- Models with insufficient capacity are unable to solve complex tasks.
- Models with high capacity can solve complex tasks,
 - but when their capacity is higher than they may overfit.
- We compare a linear, quadratic and degree-9 predictor attempting to fit a problem where the true underlying function is quadratic.
- The linear function is **unable to capture the curvature** in the true underlying problem, so it underfits.
- The **degree-9 predictor** is capable of representing the correct function, but it is also capable of representing infinitely many other functions that pass exactly through the training points,

Capacity, Overfitting and Underfitting



(Left) A linear function fit to the data suffers from underfitting—it cannot capture the curvature that is present in the data.

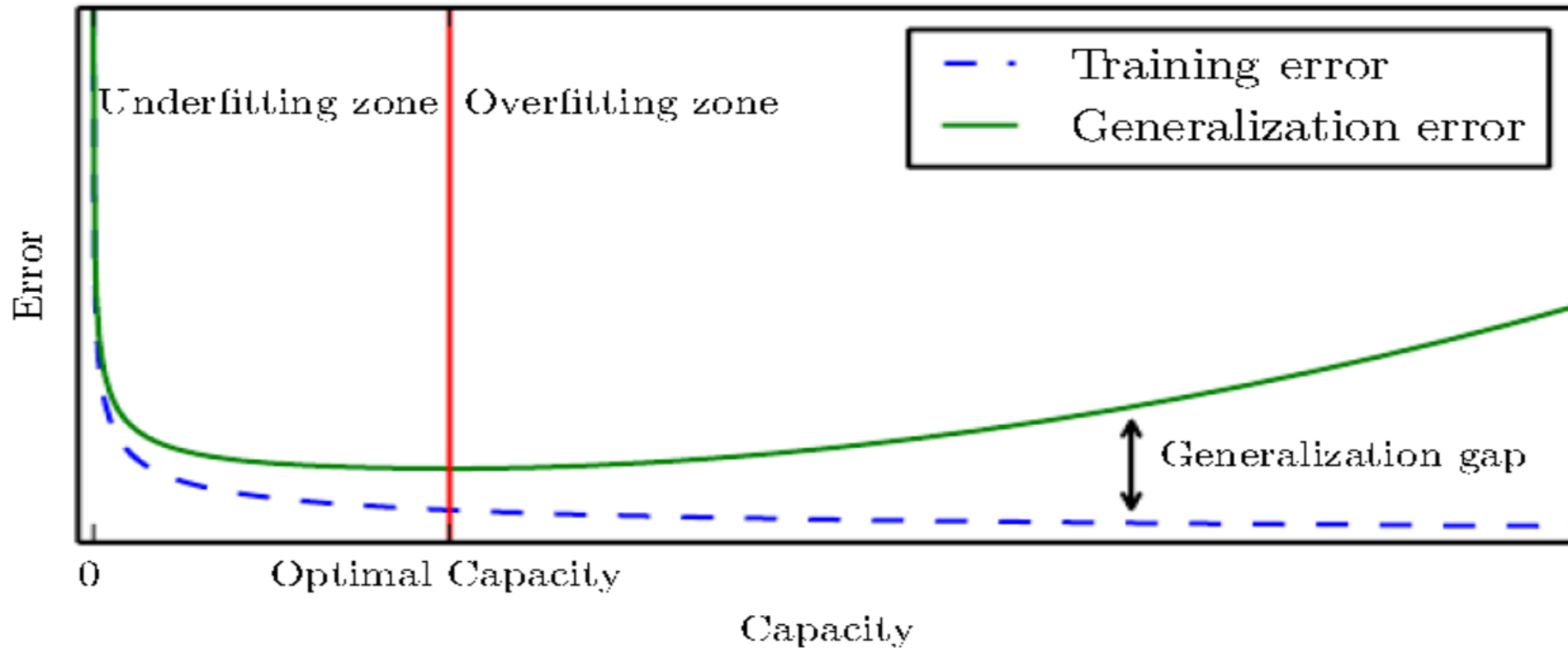
(Center) A quadratic function fit to the data generalizes well to unseen points. It does not suffer from a significant amount of overfitting or underfitting.

(Right) A polynomial of degree 9 fit to the data suffers from overfitting

Capacity, Overfitting and Underfitting

- one way of changing a model's capacity:
- by changing the number of input features it has,
- and simultaneously adding new parameters associated with those features.
- In many cases, finding the best function within this family is a very difficult optimization problem.
- learning algorithm does not actually find the best function, but merely one that significantly reduces the training error.

Capacity, Overfitting and Underfitting



Capacity, Overfitting and Underfitting

- Training and test error behave differently.
- At the left end of the graph, **training error and generalization error are both high. This is the underfitting regime.**
- **As we increase capacity, training error decreases, but the gap between training and generalization error increases.**

Capacity, Overfitting and Underfitting

- **Non-parametric models have no such limitation.**
- One example of such an algorithm is **nearest neighbor regression**.
- Unlike linear regression, which has a fixed-length vector of weights,
- the nearest neighbor regression model simply stores the **X and y from the training set**.
- When asked to classify a test point x , the model looks up the nearest entry in the training set and returns the associated regression target.
- $\hat{y} = y_i$ **where $i = \arg \min \|X_i - x\|_2^2$**

Capacity, Overfitting and Underfitting

- The ideal model is an **oracle** that simply knows the true **probability distribution that generates the data**.
- Even such a model **will still incur some error** on many problems, because there may still be **some noise in the distribution**.

The error incurred by an oracle making predictions from the true distribution $p(x, y)$ is called the **Bayes error**.

Capacity, Overfitting and Underfitting

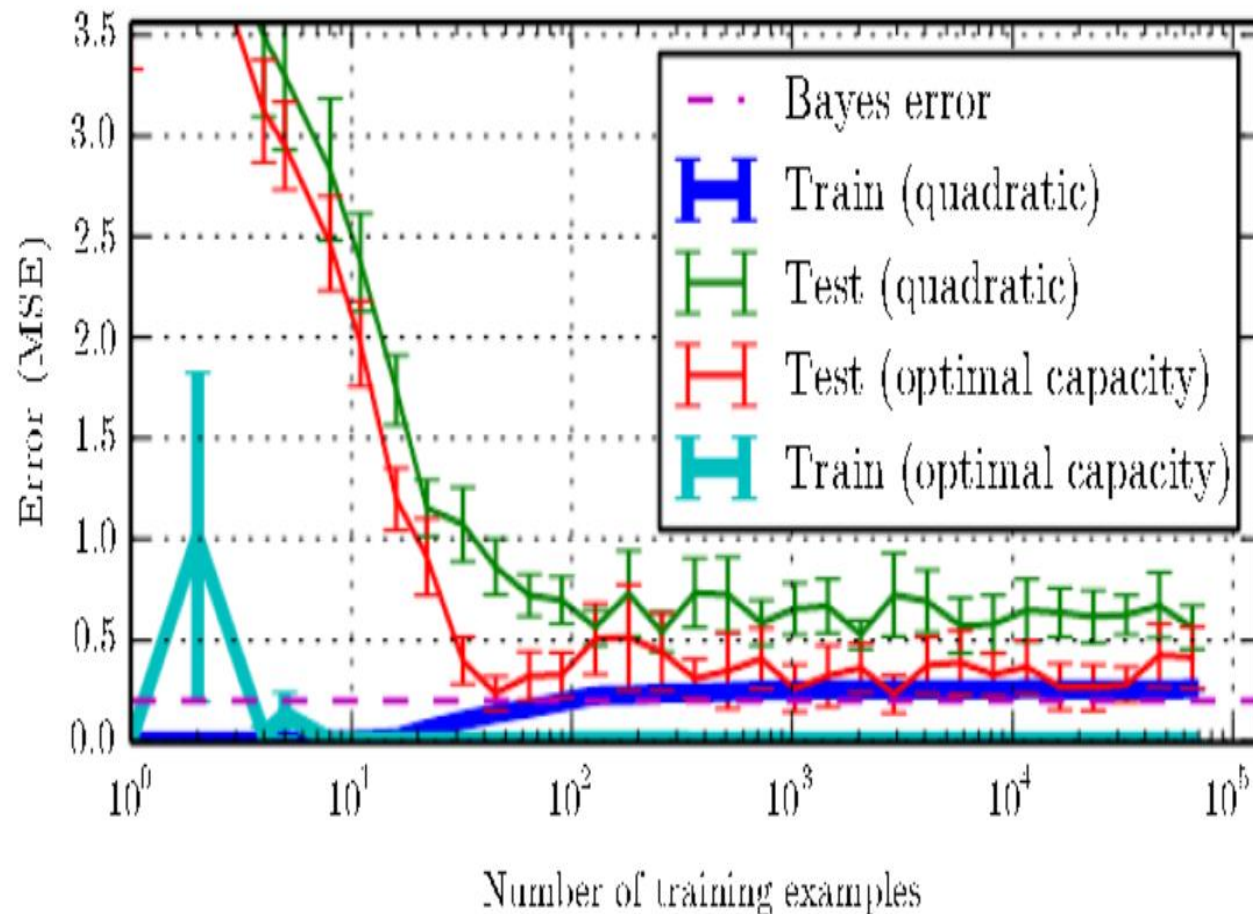
- Training and generalization error vary as the size of the training set varies.
- Expected generalization error can never increase as the number of training examples increases.
- For non-parametric models, more data yields better generalization until the best possible error is achieved.

Any fixed parametric model with less than optimal capacity will asymptote to an error value that exceeds the Bayes error.

Capacity, Overfitting and Underfitting

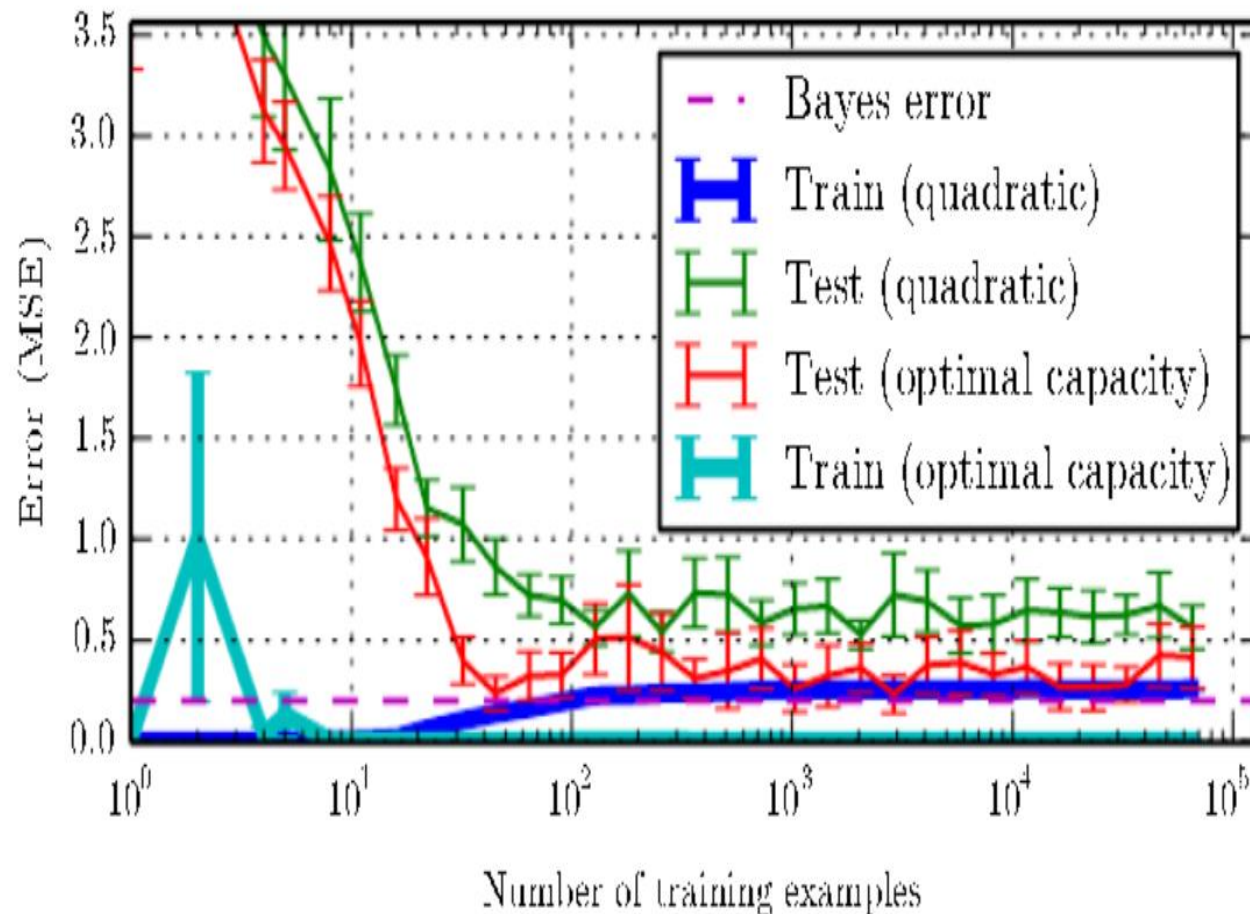
- it is possible for the model to have **optimal capacity** and yet still have a **large gap between training and generalization error**.
- we may be able to reduce this gap by gathering more training examples.
- **The No Free Lunch Theorem**
- every classification algorithm has the **same error rate** when classifying previously unobserved points.
- **no machine learning algorithm is universally any better than any other.**

Capacity, Overfitting and Underfitting



For the quadratic model,
training error increases as size of the training set increases.
test error decreases, because fewer incorrect hypotheses are consistent with the training data.
The quadratic model **does not have enough capacity** to solve the task, so its test error asymptotes to a **high value**.

Capacity, Overfitting and Underfitting



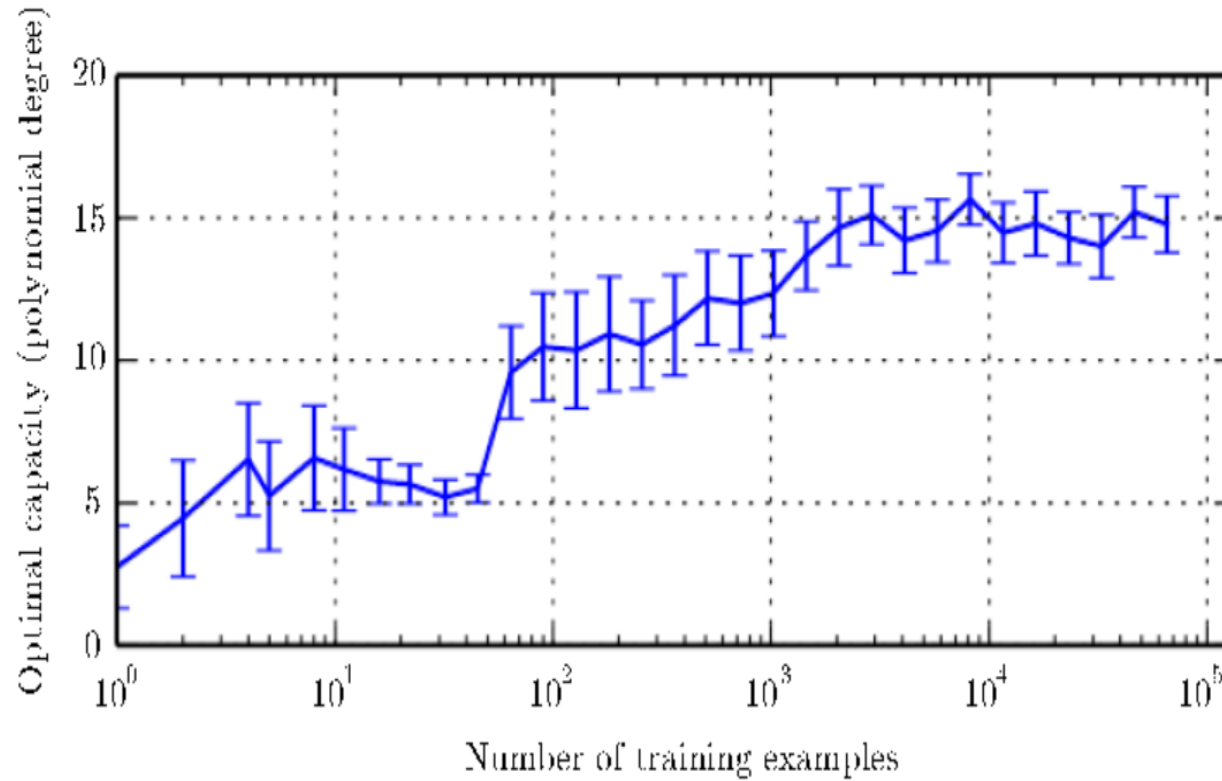
test error at optimal capacity

asymptotes to Bayes error.

training error can fall below **Bayes error**,

As **training size** increases to **infinity**,
training error of any fixed-capacity
model **quadratic model**) must rise to
at least Bayes error

Capacity, Overfitting and Underfitting



As the training set size increases, the optimal capacity (degree of the optimal polynomial regressor) increases.

goal of machine learning research is not to seek a universal learning algo or absolute best learning algo. Instead, our goal is to understand what kinds of distributions are relevant to the “real world” .

Capacity, Overfitting and Underfitting

- **Regularization**

- no free lunch theorem implies that we must design our machine learning algorithms to perform well on a specific task.
- **We do so by building a set of preferences into the learning algorithm.**
- linear functions can be very useful for problems where the relationship between inputs and outputs truly is close to linear.
- They are less useful for problems that behave in a very nonlinear fashion.

For example,

linear regression not perform very well if we tried to use it to predict **$\sin(x)$** from **x** .

We can **control the performance of** our algorithms by choosing what kind of functions we allow them to draw solutions from,
as well as by controlling the amount of these functions.

Capacity, Overfitting and Underfitting

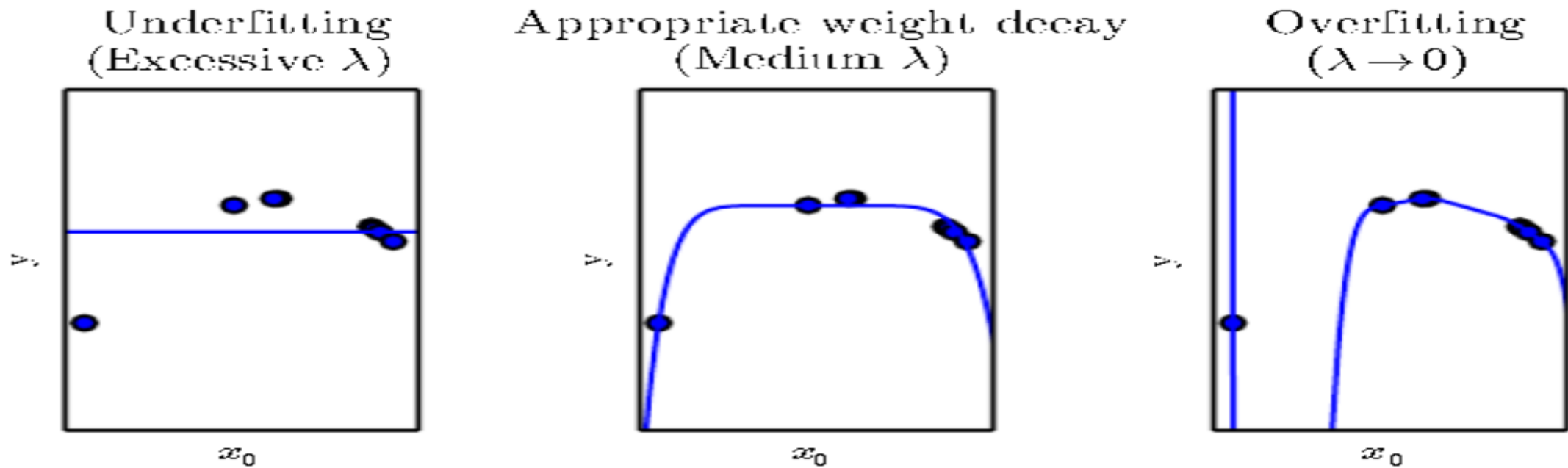
- **Regularization**

- we can modify the training criterion for linear regression to **include weight decay**.

$$J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^T \mathbf{w}$$

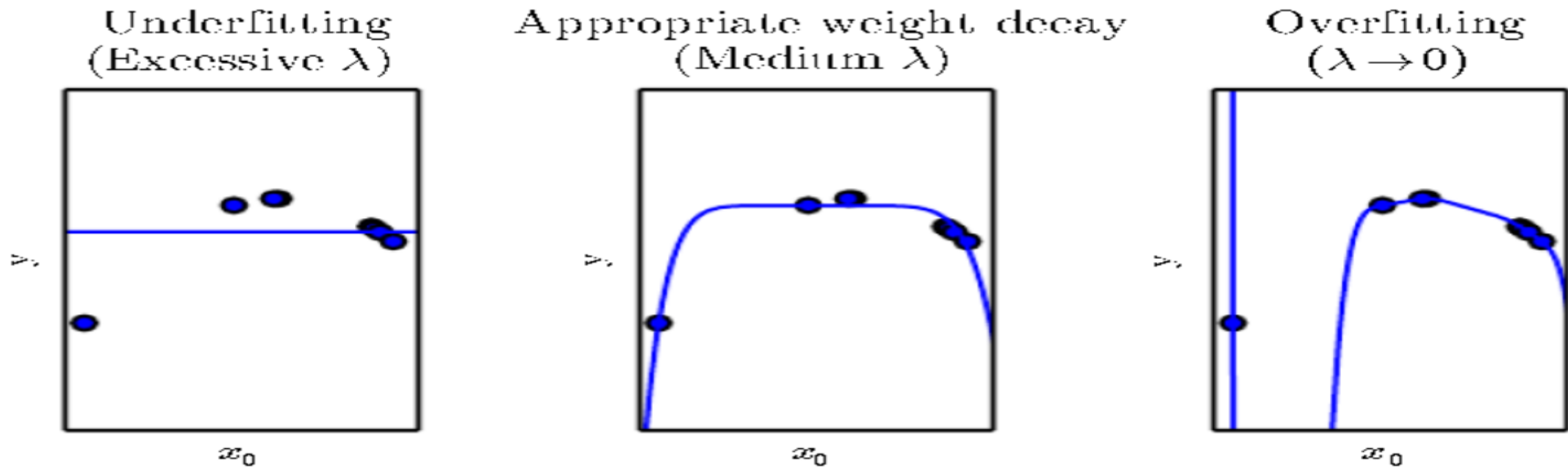
- where λ is a value that controls the strength of our preference for smaller weights.
- When $\lambda = 0$, we impose no preference,
- and **larger λ forces the weights to become smaller**.
- we can control a model's tendency to overfit or underfit via weight decay,
- we can train **high-degree polynomial** regression model **with different values of λ** .

Capacity, Overfitting and Underfitting



- **we use only models with degree 9.**
- (Left) With very large λ , we can force the model to learn a function with no slope at all.
- This underfits because it can only represent a constant function

Capacity, Overfitting and Underfitting



- *(Center)* With a medium value of λ , the learning algorithm recovers a curve with the right general shape.
- *(Right)* With weight decay approaching zero, the degree-9 polynomial overfits significantly

Capacity, Overfitting and Underfitting

- we can **regularize** a model that learns a function $f(x; \vartheta)$ by adding a penalty called a regularizer to the cost function.
- **In the case of weight decay, the regularizer is $\Omega(\mathbf{w}) = \mathbf{w}^T \mathbf{w}$.**
- *Regularization is any modification we make to a learning algorithm that is intended to reduce its **generalization error but not its training error**.*
- The no free lunch theorem has made it clear that there is no best machine learning algorithm, and no best form of regularization.

Instead we must choose a form of regularization that is well-suited to the particular task we want to solve

Hyperparameters and Validation Sets

- Most machine learning algorithms have **several settings**
- that we can use to control the behavior of the learning algorithm.
- These settings are called **hyperparameters**.
- **degree of the polynomial**, which acts as a **capacity hyperparameter**.
- **λ value** used to **control strength** of **weight decay** is example of **hyperparameter**
- it is not appropriate to learn that hyperparameter on the training set

Hyperparameters and Validation Sets

- Specifically, we split the **training data into two disjoint subsets**.
- One of **these subsets** is used to **learn the parameters**.
- The other subset is our **validation set**, used to estimate **the generalization error** during or after training, allowing for the **hyperparameters to be updated accordingly**.
- Typically, **one uses about 80% of the training data** for training
- **and 20% for validation**.
- Since the **validation set** is used to **“train” the hyperparameters**,
- the validation set error will underestimate the generalization error, by a smaller amount than the training error.
- After all hyperparameter optimization is complete, the generalization error may be estimated using the test set.

Hyperparameters and Validation Sets

- **A) Cross-Validation**

- Dividing the dataset into a **fixed training set** and a **fixed test set** can be problematic if it results in the **test set being small**.
- A small test set implies statistical uncertainty around the estimated average test error, making it difficult to claim that algorithm *A* works better than algorithm *B* on the given task.
- When the dataset has **hundreds of thousands of examples or more**, this is not a serious issue.
- **We get problems when the dataset is too small**, estimation of the mean **test error**, at the price of increased computational cost.

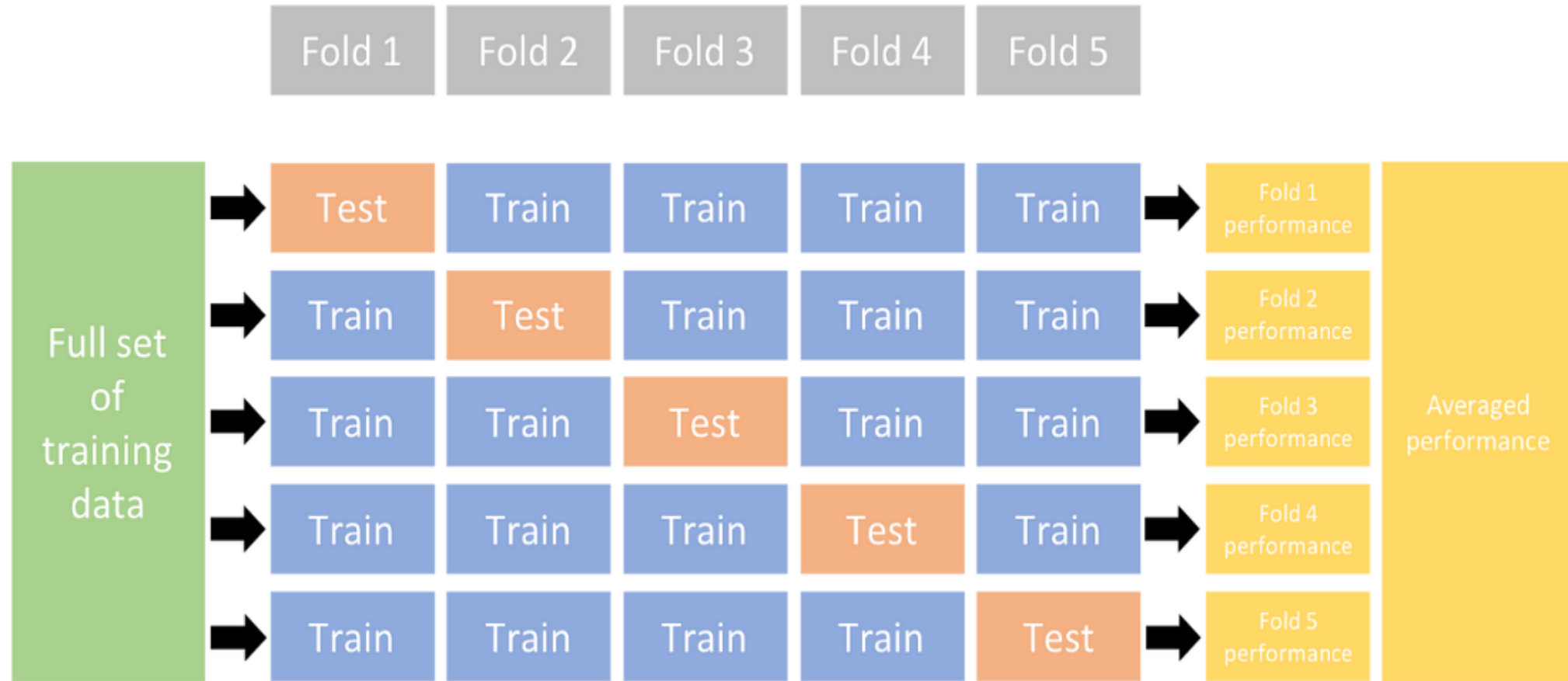
Hyperparameters and Validation Sets

A) Cross-Validation

- These procedures are based on the idea of repeating the training and testing computation on different randomly chosen subsets or splits of the original dataset.
- The most common of these is the ***k*-fold cross-validation** procedure

Hyperparameters and Validation Sets

A) Cross-Validation



Hyperparameters and Validation Sets

A) Cross-Validation

- **k -fold cross-validation** procedure.
- in which a partition of the dataset is formed by splitting it into **k non-overlapping subsets**.
- The test error may then be estimated by taking the average test error across k trials.
- On trial i , the i -th subset of the data is used as the test set and the rest of the data is used as the training set.

Estimators, Bias and Variance

- **parameter estimation, bias and variance** are useful to **generalization, underfitting and overfitting.**
- **Point Estimation**
 - attempt to provide the single “best” prediction of some quantity of interest.
 - can be **single parameter or a vector of parameters** in some parametric model,
 - such as the **weights in our linear regression** but it can also be a **whole function.**
 - denote a point estimate of a parameter θ by $\hat{\theta}$
 - .

Estimators, Bias and Variance

- **Point Estimation**
- **Let** $\{\mathbf{x}^{(1)}, \dots \dots \mathbf{x}^{(m)}\}$ be a set of m independent and identically distributed (i.i.d.) data points.
- A point estimator or statistic function of the data:
- $\widehat{\boldsymbol{\theta}}_m = \mathbf{g}(\mathbf{x}^{(1)}, \dots \dots \mathbf{x}^{(m)})$
- a good estimator is a function whose output is **close to the true** underlying $\boldsymbol{\theta}$ that generated the training data

Estimators, Bias and Variance

- **Point Estimation**
- we assume that the true parameter value θ is fixed but unknown, while the point estimate $\hat{\theta}$ is a function of the data.
- Since the data is drawn from a random process, any function of the data is random. Therefore $\hat{\theta}$ is a random variable.
- Point estimation can also refer to the estimation of the **relationship between input and target variables**

Estimators, Bias and Variance

- **Function Estimation**

- we are trying to predict a **variable y given an input vector x** .
- For example,
- we may assume that $y = f(x) + \epsilon$, where ϵ stands for the part of y that is not predictable from x .
- In function estimation, we are interested in approximating f with a model or estimate $\hat{f}$.

Estimators, Bias and Variance

- **Function Estimation**
- The linear regression example and the polynomial regression example are both examples of scenarios that may be interpreted either as **estimating a parameter w**
- **or estimating a function $\hat{f}$ mapping from x to y .**

Bias

- $\text{bias}(\hat{\theta}_m) = E(\hat{\theta}_m) - \theta$
- θ is the true underlying value of θ used to define the data generating distribution.
- $\hat{\theta}_m$ is said to be **unbiased** if $\text{bias}(\hat{\theta}_m) = 0$,
- which implies $E(\hat{\theta}_m) = \theta$

Bernoulli Distribution

- Consider a set of samples $\{x^{(1)}, \dots, x^{(m)}\}$
- that are independently and identically distributed according to a Bernoulli distribution with **mean θ** :
- $P(x^{(i)}; \theta) = \theta^{x^{(i)}} (1 - \theta)^{(1-x^{(i)})}$
- $\widehat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$
- $bias(\widehat{\theta}_m) = E(\widehat{\theta}_m) - \theta$
- $= E\left[\frac{1}{m} \sum_{i=1}^m x^{(i)}\right] - \theta$
- $\frac{1}{m} \sum_{i=1}^m E[x^{(i)}] - \theta$

Bernoulli Distribution

- $= E \left[\frac{1}{m} \sum_{i=1}^m x^{(i)} \right] - \theta$
- $\frac{1}{m} \sum_{i=1}^m E[x^{(i)}] - \theta$
- $\frac{1}{m} \sum_{i=1}^m \sum_{x^{(i)}=0}^1 (x^{(i)} \theta^{x^{(i)}} (1 - \theta)^{(1-x^{(i)})} - \theta)]$
- $\frac{1}{m} \sum_{i=1}^m (\theta) - \theta$
- $= \theta - \theta = 0$
- Since $\text{bias}(\widehat{\theta}_m) = 0$, we say that our estimator $\widehat{\theta}$ is unbiased

Gaussian Distribution Estimator of the Mean

- Consider a set of samples $\{x^{(1)}, \dots, x^{(m)}\}$
- that are independently and identically distributed according to a **Gaussian distribution**
- $P(x^{(i)}) = N(x^{(i)}; \mu, \sigma^2)$;
- where $i \in \{1, \dots, m\}$.
- Gaussian probability density function is given by
- $p(x^{(i)}; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(x^{(i)} - \mu)^2}{\sigma^2}\right)$
- $\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$

Gaussian Distribution Estimator of the Mean

- $\text{bias}(\hat{\mu}_m) = E(\hat{\mu}_m) - \mu$
- $= E\left[\frac{1}{m} \sum_{i=1}^m x^{(i)}\right] - \mu$
- $\frac{1}{m} \sum_{i=1}^m E[x^{(i)}] - \mu$
- $\frac{1}{m} \sum_{i=1}^m (\mu) - \mu$
- $= \mu - \mu = 0$
- Since $\text{bias}(\hat{\mu}_m) = 0$, we say that our estimator $\hat{\mu}$ is unbiased.
- mean is an unbiased estimator of Gaussian mean parameter.

Estimators of Variance of a Gaussian Distrib

- The first estimator of σ^2 we consider is known as the **sample variance**
- $\hat{\mu}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$
- $\hat{\sigma}_m^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$
- ***bias*** $(\hat{\sigma}_m^2) = E(\hat{\sigma}_m^2) - \sigma^2$
- $E(\hat{\sigma}_m^2) = E[\frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2]$
- $= \frac{m-1}{m} \sigma^2$
- we conclude that the bias of $\hat{\sigma}_m^2$ is $-\sigma^2/m$, Therefore,
- the sample variance is a **biased estimator**.

The unbiased sample variance estimator

- $\hat{\sigma}_m^2 = \frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2$
- $E(\hat{\sigma}_m^2) = \sigma^2$
- $E(\hat{\sigma}_m^2) = E[\frac{1}{m-1} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2]$
- $= \frac{m}{m-1} E(\hat{\sigma}_m^2)$

Since $E(\hat{\sigma}_m^2) = E[\frac{1}{m} \sum_{i=1}^m (x^{(i)} - \hat{\mu}_m)^2] = \frac{m-1}{m} \sigma^2$

- $= \frac{m}{m-1} (\frac{m-1}{m} \sigma^2)$
- $= \sigma^2$

- We have two estimators: one is biased and the other is not.
- While unbiased estimators are clearly desirable, they are not always the “best” estimators

Variance and Standard Error

- how much we expect it to vary as a function of the data sample.
- The variance of an estimator is simply the variance. $\text{Var}(\hat{\theta})$
- where the random variable is the training set.
- square root of variance is called standard error denoted $\text{SE}(\hat{\theta})$.
- estimator to exhibit relatively low variance.
- $\text{SE}(\hat{\mu}_m) = \sqrt{\text{Var} \left[\frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)} \right]} = \frac{\sigma}{\sqrt{m}}$
- where σ^2 is the true variance of the samples $\mathbf{x}^{(i)}$.

Variance and Standard Error

- neither the square root of the sample variance nor the square root of the unbiased estimator of the variance provide an **unbiased estimate of the standard deviation**.
- Both approaches tend to **underestimate the true standard deviation**, but are still used in practice.
- The standard error of the mean is very useful in machine learning experiments.
- We often estimate the generalization error by computing the sample mean of the error on the test set

Variance and Standard Error

- **we can use the standard error to compute the probability** that the true expectation falls in any chosen interval.
- **For example, the 95% confidence interval centered on**
- the mean $\hat{\mu}_m$ is
- $(\hat{\mu}_m - 1.96SE(\hat{\mu}_m), \hat{\mu}_m + 1.96SE(\hat{\mu}_m))$,
- under the normal distribution with mean $\hat{\mu}_m$ and variance $SE(\hat{\mu}_m)^2$
-

Variance and Standard Error

- Bernoulli Distribution
- $\widehat{\theta}_m = \frac{1}{m} \sum_{i=1}^m x^{(i)}$
- $\text{Var}(\widehat{\theta}_m) = \text{Var}\left(\frac{1}{m} \sum_{i=1}^m x^{(i)}\right)$
- $= \frac{1}{m^2} \sum_{i=1}^m \text{Var}(x^{(i)})$
- $= \frac{1}{m^2} \sum_{i=1}^m \theta(1 - \theta)$
- $= \frac{1}{m^2} m \theta(1 - \theta)$
- $= \frac{1}{m} \theta(1 - \theta)$
- The variance of the estimator decreases as a function of m , the number of examples in the dataset

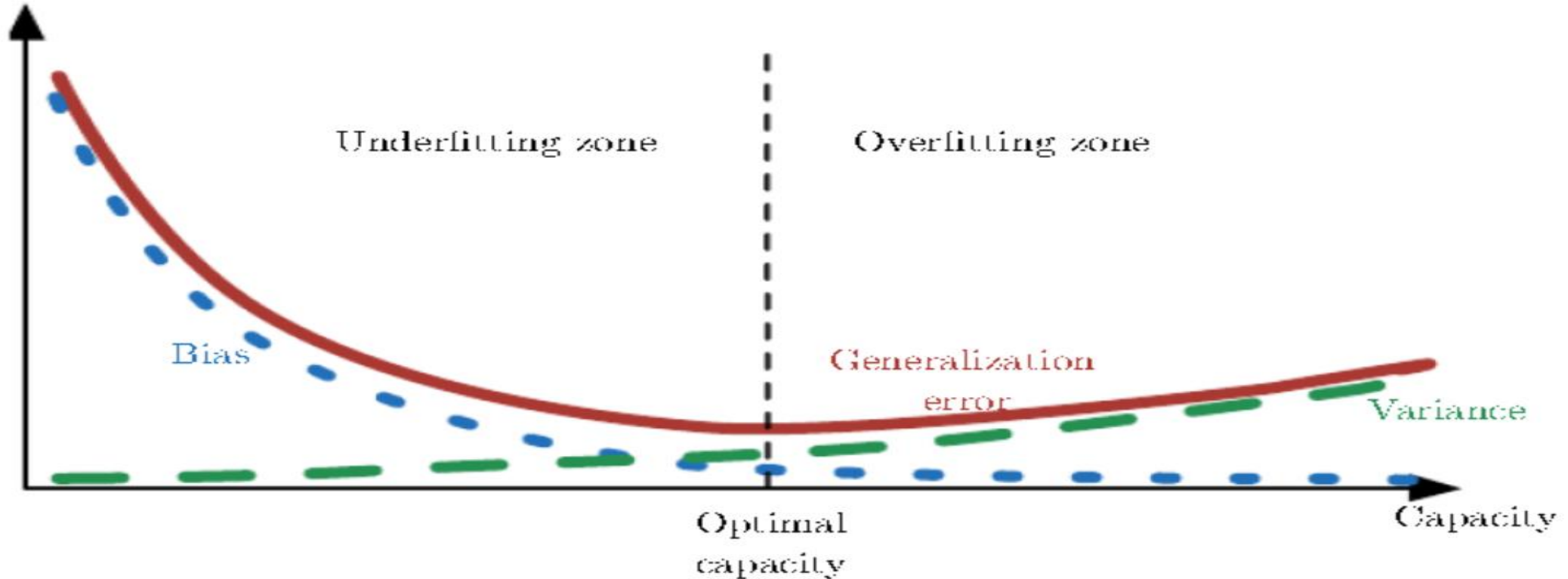
Trading off Bias and Variance to Minimize Mean Squared Error

- **Bias** is defined as the inability of the model because of that there is **some difference or error** occurring between the **model's predicted value and the actual value**.
- **Bias and variance** measure two different **sources of error in an estimator**.
- Bias measures the expected **deviation from the true value** of the function or parameter.
- Variance provides a **measure of the deviation from the expected estimator** value that any particular sampling of the data is likely to cause.
- What happens when we are given a choice between two estimators, **one with more bias and one with more variance**?
- How do we choose between them?
- The most common way to negotiate this trade-off is to use **cross-validation**.

Trading off Bias and Variance to Minimize Mean Squared Error

- we can also compare the mean squared error (MSE) of the estimates.
- $MSE = E [(\widehat{\theta}_m - \theta)]^2$
- $= Bias(\widehat{\theta}_m)^2 + Var(\widehat{\theta}_m)$
- evaluating the MSE incorporates both **bias and the variance**.
- Desirable estimators are those with small MSE.
- and these are estimators that manage to keep both their bias and variance somewhat in check

Trading off Bias and Variance to MSE error



- As capacity increases (x-axis), bias (dotted) tends to decrease and variance (dashed) tends to increase, yielding another U-shaped curve for generalization error (bold curve).
- If we vary capacity along one axis, there is an optimal capacity, with underfitting when the capacity is below this optimum and overfitting when it is above.